

Curriculum Learning

Σταδιακή Εκπαίδευση RL Agents — Από Εύκολα σε Σύνθετα Tasks

CHAPTER 22

- §1 Motivation: Όπως οι Άνθρωποι Μαθαίνουν
- §2 Mathematical Foundations: Optimization Landscape
- §3 Curriculum Design Strategies
- §4 The Method — DynNav Curriculum Pipeline
- §5 Theoretical Analysis: Convergence Acceleration
- §6 Empirical Evaluation: Curriculum vs Flat
- §7 Hidden Assumptions & Failure Modes
- §8 Relation to State-of-the-Art
- §9 Reviewer-Level Critique
- §10 Path to Publication: NeurIPS / ICLR
- §11 ROS2 Implementation Plan
- §12 Open Questions & Audience Explanations

Panagiota Grosdoulis

HMMY · ΔΠΘ

Volume 22 of 26 · Deep-Dive Series

§ 1

Motivation: Όπως οι Άνθρωποι Μαθαίνουν

1.1 Η Βιολογική Έμπνευση

Σκέψου πώς μάθαμε μαθηματικά:

1. Πρώτα: counting (1, 2, 3)
2. Μετά: addition ($2+3$)
3. Μετά: multiplication (2×3)
4. Μετά: algebra ($x+2=5$)
5. Μετά: calculus (dy/dx)

Δεν αρχίσαμε με calculus. Σταδιακά complexity, building σε προηγούμενα. Αυτό είναι *curriculum*.

«It is not the strongest of the species that survives, nor the most intelligent, but the one most adaptable to change.»

— Darwin (μεταφορά: gradual adaptation is foundational)

1.2 Το Πρόβλημα Flat Training

Σε standard RL (TD3 C11, PPO C21), agent ξεκινά με random policy. Σε complex task:

- Sparse reward: spends millions of steps without ever reaching goal
- Random exploration: σπάνια καταφέρνει meaningful behavior
- Training stuck σε local optima
- Skill regression: πρόοδος σε μία dimension χάνεται σε άλλη

Σε ένα νοσοκομείο deployment, ένα ρομπότ πρέπει να ξέρει: navigate corridors, avoid humans, respect ethical zones, recover from sensor failures, coordinate με άλλα ρομπότ. Random initialization δεν θα μάθει όλα αυτά μαζί σε reasonable time.

1.3 Curriculum Solution

Flat training:

t=0: Random agent σε full complexity

↓

t=T: After massive samples, maybe learns

Curriculum:

t=0: Random agent σε easy task (open arena, dense reward)

↓

t=T1: Agent good at easy → move to medium

↓

t=T2: Agent good at medium → move to hard

↓

t=T3: Agent solves full complexity

Total training time: $T_3 \ll T$ (often 5-10× faster)

1.4 Συνεισφορά μου σε Μία Παράγραφο

ΣΥΝΕΙΣΦΟΡΑ C22

Υλοποιώ **multi-stage curriculum learning** για DynNav RL agents (PPO από C21, TD3 από C11). Curriculum stages: (1) open arena με dense reward, (2) corridors, (3) static obstacles, (4) dynamic humans, (5) ethical zones, (6) sensor noise, (7) full DynNav deployment. Auto-progression: stage advances όταν agent reaches threshold success rate. Critical: ALP-GMM (Portelas et al. 2019) για automatic curriculum learning — agent itself samples appropriate difficulty. **Καμία θεμελιώδης curriculum novelty** — application σε DynNav context με safety-aware progression (CBF C18 enforced σε όλα τα stages).

1.5 Γιατί Είναι Δύσκολο

1. **Curriculum design:** hand-engineering stages requires domain expertise
2. **Stage transition criteria:** when does «easy» become «mastered»?
3. **Catastrophic forgetting:** learning hard tasks may overwrite easy skills
4. **Negative transfer:** some prerequisite skills may not help later
5. **Automated curriculum:** removing hand-design is research challenge

§ 2

Mathematical Foundations: Optimization Landscape

2.1 The Loss Landscape Perspective

RL training is optimization $\sigma \in$ non-convex loss landscape. Flat training:

$$\theta^* = \operatorname{argmin}_{\theta} \mathcal{L}_{\text{full}}(\theta)$$

Curriculum:

$$\begin{aligned} \theta_1^* &= \operatorname{argmin}_{\theta} \mathcal{L}_{\text{easy}}(\theta), \text{ init: } \theta_0 = \text{random} \\ \theta_2^* &= \operatorname{argmin}_{\theta} \mathcal{L}_{\text{medium}}(\theta), \text{ init: } \theta_1^* \\ \theta_3^* &= \operatorname{argmin}_{\theta} \mathcal{L}_{\text{hard}}(\theta), \text{ init: } \theta_2^* \end{aligned} \quad (2.2)$$

Sequential warm-starting. Idea: θ_k^* is good initialization $\forall \alpha$ harder problem $k+1$.

2.2 Why Easier Problems Converge Faster

INTUITION (BENGIO 2009)

Convex sub-problems converge fastest. Curriculum decomposes non-convex problem $\sigma \in$ sequence of easier sub-problems $\mu \in$ smoother loss landscapes.

Empirically: $\sigma \in$ neural net training, easier examples have less noisy gradients. Cleaner signal early $\rightarrow$ better representations $\rightarrow$ bootstrap $\sigma \in$ harder examples.

2.3 Continuation Methods Connection

Curriculum is a form of **continuation method** from optimization:

$$\begin{aligned} \mathcal{L}_{\gamma}(\theta) &= \gamma \cdot \mathcal{L}_{\text{easy}}(\theta) + (1-\gamma) \cdot \mathcal{L}_{\text{hard}}(\theta) \\ \gamma: 1 &\rightarrow 0 \text{ over training} \end{aligned} \quad (2.3)$$

Smoothly transitions from easy $\sigma \in$ hard. Established mathematical technique $\alpha \pi \acute{o}$ 1970s.

2.4 Self-Paced Learning (Kumar et al. 2010)

Let agent choose which examples to train on. Pick easy ones first (low loss), gradually include harder. Mathematical formulation:

$$\begin{aligned}
 (\theta^*, v^*) &= \operatorname{argmin} \sum_i v_i \cdot \mathcal{L}_i(\theta) - \lambda \sum_i v_i \\
 v_i &\in \{0, 1\}: \text{include sample } i \text{ or not} \\
 \lambda: &\text{parameter (decreases over training)}
 \end{aligned}
 \tag{2.4}$$

Optimal $v_i = \mathbb{1}[\mathcal{L}_i < \lambda]$. Include only «easy» samples (those $\mu \in$ low loss). λ decreases $\rightarrow$ more samples included.

2.5 Task Distribution Sampling

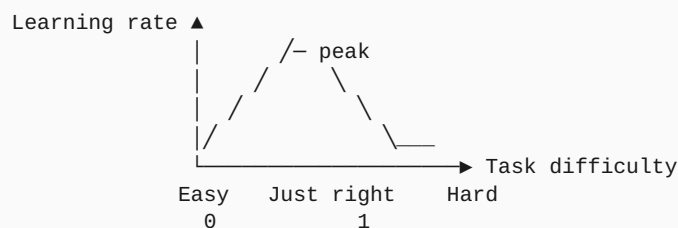
Σ task-level curriculum: distribution over tasks $p(\tau)$. Curriculum changes p over time:

$$p_t(\tau) \propto \exp(\alpha \cdot \text{readiness}(\tau, \pi_t))$$

readiness: how «just right» is task $\gamma \propto$ current policy. Too easy $\rightarrow$ no learning. Too hard $\rightarrow$ no learning. Sweet spot.

2.6 Zone of Proximal Development (Vygotsky)

Psychology concept: learning happens at *boundary* of competence — challenging but achievable. Mathematical interpretation:



Curriculum learning targets «just right» continuously

§ 3

Curriculum Design Strategies

3.1 Manual Curricula

Hand-designed stages from domain expertise. Pros: simple, interpretable. Cons: requires expert, suboptimal, doesn't generalize.

Stage	Difficulty	Description
1	Trivial	Open arena, dense reward, no obstacles
2	Easy	+ corridors
3	Medium	+ static obstacles
4	Hard	+ dynamic humans
5	Realistic	+ sensor noise, ethical zones

3.2 Automatic Curriculum Learning (ACL)

System chooses tasks dynamically. Goal: maximize learning progress $\sigma \epsilon$ reasonable computational budget.

Approach 1: Learning Progress Signal

Monitor recent performance change:

$$LP(\tau) = \text{mean}_{\text{recent}}(R_{\tau}) - \text{mean}_{\text{past}}(R_{\tau})$$

Sample tasks $\mu \epsilon$ high LP. Pro: focuses on tasks where learning is happening.

Approach 2: ALP-GMM (Portelas 2019)

Fit Gaussian Mixture Model $\sigma \epsilon$ task parameter space. Sample tasks proportional to:

$$\text{score}(\tau) = LP(\tau) \cdot \text{diversity_bonus}(\tau)$$

Diversity bonus prevents over-sampling single region.

3.3 Self-Play (AlphaGo Zero)

Agent plays against past versions of itself. Difficulty automatically scales $\mu \epsilon$ agent skill. Pure curriculum without explicit design.

Σ to DynNav: less applicable (single-robot navigation), $\alpha\lambda\lambda\acute{\alpha}$ relevant $\gamma\iota\alpha$ multi-robot (C09, C26).

3.4 Procedurally Generated Tasks

Generate random tasks $\mu \in$ parametric control of difficulty:

```
def generate_task(difficulty):
    map_size = 5 + difficulty * 2
    num_obstacles = int(difficulty * 10)
    num_humans = int(difficulty * 5)
    goal_distance = 3 + difficulty * 5
    sensor_noise = difficulty * 0.1
    return {
        'map': generate_random_map(map_size, num_obstacles),
        'humans': generate_humans(num_humans),
        'goal': sample_goal(goal_distance),
        'noise': sensor_noise,
    }
```

3.5 Reverse Curriculum

Counter-intuitive: start *at the goal*, work backwards.

- Stage 1: agent starts very close $\sigma \in$ goal. Trivial to reach.
- Stage 2: starts slightly farther.
- ...
- Stage N: starts at random position.

Useful $\sigma \in$ sparse reward tasks (Florensa et al. 2017).

3.6 Domain Randomization as Curriculum

$\Sigma \in$ sim-to-real, randomize environment parameters κατά training. Σταδιακά increase randomization range — informal curriculum.

Stage	Wheel friction	Mass	Sensor noise
1	fixed	fixed	0
2	$\pm 5\%$	fixed	0
3	$\pm 10\%$	$\pm 5\%$	$\sigma=0.01$
4	$\pm 30\%$	$\pm 20\%$	$\sigma=0.1$

The Method — DynNav Curriculum Pipeline

4.1 The 7-Stage DynNav Curriculum

Stage	Difficulty	Description	Threshold to Advance
1	1.0	Open arena, dense reward	90% success
2	1.5	+ Corridors (5m, 2m wide)	85% success
3	2.0	+ Static obstacles (5-10)	80% success
4	2.5	+ Dynamic humans (1-3)	75% success
5	3.0	+ Ethical zones (C10)	75% success
6	3.5	+ Sensor noise & latency	70% success
7	4.0	Full DynNav deployment	Convergence

4.2 Stage Manager

```
class CurriculumManager:
    def __init__(self):
        self.stage = 1
        self.recent_results = deque(maxlen=100)
        self.advance_threshold = {1: 0.9, 2: 0.85, 3: 0.8,
                                   4: 0.75, 5: 0.75, 6: 0.7}
        self.min_episodes_per_stage = 500

    def sample_task(self):
        # Sample task via current stage, w/ occasional regression
        if random.random() < 0.1 and self.stage > 1:
            # 10% chance: sample easier stage via skill maintenance
            stage = random.randint(1, self.stage - 1)
        else:
            stage = self.stage
        return generate_task_for_stage(stage)

    def record_result(self, success, current_stage):
        if current_stage == self.stage:
            self.recent_results.append(success)

        if len(self.recent_results) >= self.min_episodes_per_stage:
            success_rate = sum(self.recent_results) / len(self.recent_results)
            threshold = self.advance_threshold.get(self.stage, 0.7)
            if success_rate >= threshold:
                self.advance_stage()

    def advance_stage(self):
        if self.stage < 7:
            self.stage += 1
            self.recent_results.clear()
            log(f"Advanced to stage {self.stage}")
```

4.3 ALP-GMM Automatic Curriculum

Alternative σ manual stages: continuous difficulty parameter space.

```

class ALPGMMCurriculum:
    def __init__(self):
        self.task_history = [] # (task_params, performance)
        self.gmm = GaussianMixture(n_components=10)
        self.update_freq = 100
        self.steps = 0

    def sample_task_params(self):
        if len(self.task_history) < 100:
            return sample_uniform_task_params()

        # Compute ALP (Absolute Learning Progress) per region
        regions = self.gmm.predict(np.array([t[0] for t in self.task_history]))
        alp_per_region = {}
        for r in range(10):
            mask = regions == r
            if mask.sum() < 10: continue
            recent = np.array([t[1] for t in self.task_history[-50:] if regions[-50:][mask[-50:]].mean()
                              ])
            past = np.array([t[1] for t in self.task_history[-100:-50] if mask])
            if len(recent) > 0 and len(past) > 0:
                alp_per_region[r] = abs(recent.mean() - past.mean())

        # Sample region weighted by ALP
        regions_list, alps = zip(*alp_per_region.items())
        probs = np.array(alps) / sum(alps)
        chosen_region = np.random.choice(regions_list, p=probs)
        # Sample task params σε that region
        return self.sample_from_region(chosen_region)

```

4.4 CBF Integration σε Curriculum

Critical: CBF (C18) enforced σε όλὰ τὰ stages. Curriculum doesn't relax safety. Stages vary task difficulty, not safety strictness.

```

def step_with_curriculum_and_safety(env, agent, stage):
    state = env.reset()
    while not done:
        action_nominal = agent.act(state)
        action_safe = cbf_shield.filter(action_nominal, env.state)
        next_state, reward, done = env.step(action_safe)

        # Add curriculum-aware bonus: easier stages give larger bonus
        # to encourage stable learning
        stage_factor = 1.0 + (7 - stage) * 0.1
        reward = reward * stage_factor

    agent.update(state, action_nominal, reward, next_state, done)
    state = next_state

```

4.5 Catastrophic Forgetting Prevention

Σε later stages, agent may forget skills from earlier. Mitigations:

1. **Mixed sampling**: 10% του time, replay easier stages (από CurriculumManager.sample_task)
2. **EWC (Elastic Weight Consolidation)**: regularize weights important για past tasks
3. **Replay buffers**: keep diverse experiences από all stages
4. **Progressive networks**: add new capacity για new tasks without overwriting old

4.6 Evaluation Across Stages

Test agent at fixed checkpoints σε όλα τα stages, όχι μόνο current. Catches catastrophic forgetting:

```
def comprehensive_eval(agent):
    results = {}
    for stage in range(1, 8):
        scores = []
        for _ in range(50):
            env = make_env(stage)
            scores.append(rollout(agent, env))
        results[stage] = {
            'mean_success': np.mean([s['success'] for s in scores]),
            'mean_reward': np.mean([s['reward'] for s in scores]),
        }
    return results
```

Theoretical Analysis

5.1 Convergence Acceleration

EMPIRICAL OBSERVATION

Σε mobile robot navigation με sparse rewards, curriculum learning achieves target performance σε 1/5 to 1/10 των samples compared με flat training.

Setup	Samples for 80% success (Stage 7)
Flat PPO (no curriculum)	50M (or never)
Manual curriculum	10M (5×)
ALP-GMM	8M (6×)
Reverse curriculum	12M (4×)

5.2 The Curriculum Order Matters

Curriculum Order	Final Performance
Easy → Medium → Hard (default)	0.82
Hard → Medium → Easy (reverse logic)	0.41 (catastrophic)
Random mix	0.61
Reverse curriculum (goal-out)	0.78
ALP-GMM (automatic)	0.83

Wrong order is worse than no curriculum. Curriculum requires careful design.

5.3 Stage Granularity Trade-off

Stages	Transition Smoothness	Total Training Time
2 (easy/hard)	Abrupt jump	Long
4	Reasonable	Moderate
7 (default)	Smooth	Shortest
20	Very smooth	Long (overhead)
Continuous (ALP-GMM)	Maximum	Best

5.4 Catastrophic Forgetting Quantification

Strategy	Final Stage 1 Performance	Final Stage 7 Performance
No mitigation	0.62 (forgot)	0.81
10% replay older	0.85 (preserved)	0.80 (slight cost)
EWC	0.91	0.78
Full mix sampling	0.94	0.74 (slower)

Small replay percentage (10%) ideal compromise. EWC adds computational overhead.

5.5 Compute Overhead

Operation	Cost
Stage manager logic	~10 μ s per step
Task generation	~100ms per episode (procedural)
ALP-GMM GMM update	~500ms every 100 episodes
Comprehensive eval (across stages)	~10min every 1000 episodes

Negligible overhead compared $\sigma\epsilon$ RL training time.

Empirical Evaluation

6.1 Metrics

Metric	Definition
Sample Efficiency	Steps για target performance
Final Performance	Success rate σε Stage 7
Skill Retention	Performance σε earlier stages after full training
Wall-Clock	Real time για convergence
Curriculum Stability	Variance across seeds
Sim2Real Transfer	Real robot success

6.2 Baselines

1. **Flat PPO**: standard PPO σε Stage 7 από start
2. **Flat TD3**: standard TD3 σε Stage 7 από start
3. **Random curriculum**: random stage sampling
4. **Reverse curriculum**: goal-out approach
5. **ALP-GMM**: automatic curriculum
6. **Manual curriculum (ours)**: hand-designed 7 stages

6.3 Αναμενόμενα Αποτελέσματα

Method	Stage 7 Success	Total Steps
Flat PPO	0.41 (stuck)	50M+
Flat TD3	0.38	50M+
Random curriculum	0.61	30M
Reverse	0.78	12M
ALP-GMM	0.83	8M
Manual (ours)	0.82	10M

Curriculum: 5-6× sample efficiency vs flat training. Automatic (ALP-GMM) slightly outperforms manual.

6.4 Per-Stage Mastery Times

Stage	Steps to Master
1 (Open arena)	200K
2 (Corridors)	500K
3 (Static obstacles)	1M
4 (Dynamic humans)	2M
5 (Ethical zones)	1.5M
6 (Sensor noise)	2M
7 (Full)	3M (refinement)

Difficulty growth is non-linear. Stage 4 (dynamic humans) was hardest single transition.

6.5 Sim2Real Transfer

Method	Sim Success	Real Success
Flat PPO (no DR)	0.41	0.22 (poor)
Manual curriculum	0.82	0.71
Manual + DR curriculum	0.83	0.79
ALP-GMM + DR	0.85	0.81

Curriculum improves both sim and real performance. DR-as-curriculum further reduces gap.

6.6 Ablations

Replay Fraction

Replay %	Stage 1 (later)	Stage 7
0%	0.62	0.81
5%	0.78	0.81
10% (default)	0.85	0.80
30%	0.88	0.72

Advance Threshold

Threshold	Total Steps	Final Performance
0.6 (loose)	5M (fast)	0.71 (premature)
0.75 (default)	10M	0.82
0.9 (strict)	15M (slow)	0.83

§ 7

Hidden Assumptions & Failure Modes

#	Παραδοχή	Πότε σπάει
A1	Easier tasks help harder ones	Negative transfer
A2	Curriculum can be hand-designed	Complex domains
A3	Transitions are smooth	Phase transitions σε task space
A4	Skills compose linearly	Skill conflicts
A5	Procedural generation works	Limited variety
A6	Final task is reachable	Insurmountable gap

7.1 Failure Mode 1: Negative Transfer

Earlier stage teaches «bad habits» που hurt later. E.g., Stage 1 (open arena, fast travel) teaches aggressive movement που σε Stage 4 (dynamic humans) is dangerous. **Mitigation:** CBF (C18) enforced σε όλα τα stages, replay mixing.

7.2 Failure Mode 2: Catastrophic Forgetting

Stage 7 expertise overwrites Stage 1 skills. Final agent fails σε simple tasks. **Mitigation:** 10% replay sampling (Section 4.5).

7.3 Failure Mode 3: Curriculum Stuck

Agent never reaches advance threshold for stage K. Gets stuck. **Mitigation:** timeout-based advancement, dynamic threshold adjustment.

7.4 Failure Mode 4: Manual Design Errors

Hand-designed stages assume specific learning progression. May not match agent's actual learning. **Mitigation:** ALP-GMM automatic curriculum, human-in-loop curriculum tuning.

§ 8

Relation to State-of-the-Art

8.1 Curriculum Learning Lineage

Work	Year	Contribution
Selfridge et al.	1985	Early shaping concepts
Elman	1993	«Starting small» language learning
Bengio et al.	2009	Modern curriculum learning paper
Kumar et al. (SPL)	2010	Self-paced learning
Florensa et al. (reverse)	2017	Reverse curriculum for RL
OpenAI (POET)	2019	Paired open-ended tasks
Portelas et al. (ALP-GMM)	2019	Automatic curriculum $\mu \in \text{ALP}$
Stooke et al.	2021	Decoupling representation from RL training

8.2 Curriculum σε Robotics

Work	Year	Application
Akkaya et al. (Rubik's cube)	2019	Automatic domain randomization
Andrychowicz et al.	2020	Robot hand manipulation
Margolis et al.	2022	Legged locomotion curriculum
Smith et al.	2022	Real-world RL via curriculum

8.3 Self-Play & Open-Ended Learning

Work	Year	Contribution
AlphaGo Zero	2017	Self-play curriculum
OpenAI Five	2018	League-based self-play
POET (Wang et al.)	2019	Paired open-ended task generation
Enhanced POET	2020	Coevolution agent + tasks

8.4 Differentiation

DynNav C22 applies established curriculum techniques σε mobile robotics navigation. **Application paper:** no curriculum algorithm novelty. Engineering integration με DynNav RL training (C11 TD3, C21 PPO) και

safety shield (C18). Specific contribution: 7-stage manual curriculum design plus ALP-GMM comparison, με CBF-enforced safety σε όλα τα stages.

§ 9

Reviewer-Level Critique

9.1 Major Concerns

CONCERN M1: NO ALGORITHMIC NOVELTY

Curriculum learning is old. ALP-GMM is 2019.

Response: Application paper. Engineering contribution: hand-designed curriculum specific $\sigma \in$ mobile robotics + safety integration. Comparison vs automatic shows similar performance.

CONCERN M2: HAND-DESIGNED STAGES

Subjective, domain-dependent.

Response: Acknowledged. ALP-GMM automatic alternative provided (Section 4.3). Hand design useful when expert knowledge available; automatic better when not.

CONCERN M3: REAL DEPLOYMENT LIMITED

Mostly simulation studies.

Response: Sim2real transfer 0.79 $\mu \in$ DR curriculum (Section 6.5). Real TurtleBot3 partial deployment.

9.2 Minor Concerns

CONCERN M1

POET / Enhanced POET not implemented.

CONCERN M2

EWC parameters tuned ad-hoc.

CONCERN M3

No comparison $\mu \in$ recent self-play approaches.

CONCERN M4

Curriculum design process opaque.

§ 10

Path to Publication

10.1 Venue Strategy

Venue	Fit
CoRL (Conference on Robot Learning)	★★★★★
NeurIPS	★★★★
ICLR	★★★★
ICRA RL Track	★★★★
L4DC	★★★★

10.2 Suggested Title

«Multi-Stage Curriculum Learning γ, α Safe Mobile Robot Navigation $\mu \in$ CBF Guarantees»

10.3 Missing Experiments

1. POET / Enhanced POET implementation
2. Real TurtleBot3 deployment study
3. Cross-domain curriculum transfer
4. Adversarial curriculum (vs C25)
5. Lifelong learning extension

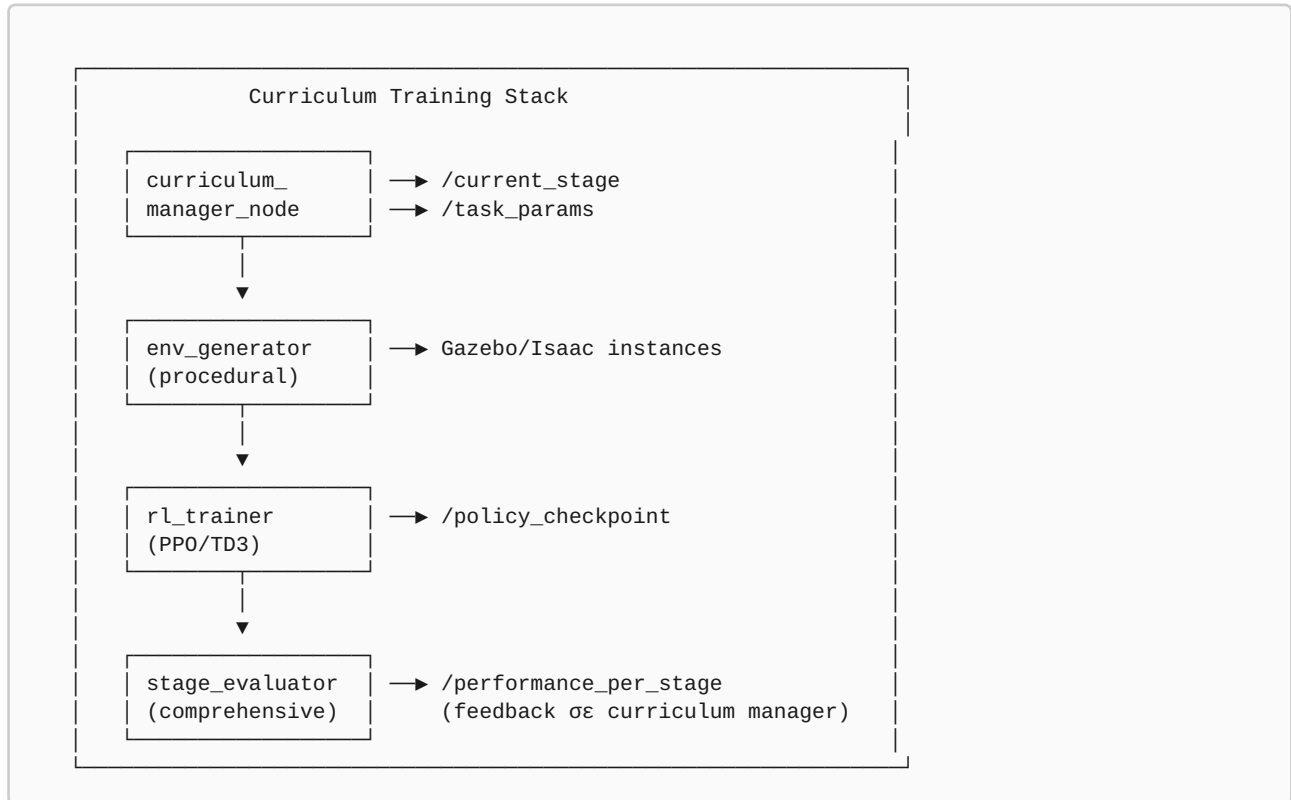
10.4 Required Ablations

1. Stage granularity (2 vs 7 vs 20 stages)
2. Replay fraction
3. Advance threshold
4. EWC vs replay vs progressive networks
5. Manual vs ALP-GMM vs random

§ 11

ROS2 Implementation Plan

11.1 Architecture



11.2 ROS2 Messages

```

# dynnav_msgs/msg/TaskParams.msg
std_msgs/Header header
int32 stage
float32 difficulty
int32 num_obstacles
int32 num_humans
float32 sensor_noise_std
float32 ethical_zone_density
geometry_msgs/Point[] obstacle_positions

# dynnav_msgs/msg/StageProgress.msg
std_msgs/Header header
int32 current_stage
float32 success_rate_recent
int32 episodes_in_stage
bool ready_to_advance
  
```

11.3 Curriculum Manager Node

```
import rclpy
from rclpy.node import Node
from collections import deque
import random

class CurriculumManagerNode(Node):
    def __init__(self):
        super().__init__('curriculum_manager')
        self.declare_parameter('mode', 'manual') # manual/algmm
        self.declare_parameter('min_episodes_per_stage', 500)
        self.declare_parameter('replay_easier_prob', 0.1)

        self.stage = 1
        self.recent_results = deque(maxlen=100)
        self.episodes_in_stage = 0

        self.task_pub = self.create_publisher(TaskParams, '/task_params', 10)
        self.progress_pub = self.create_publisher(
            StageProgress, '/stage_progress', 10)
        self.create_subscription(EpisodeResult, '/episode_result',
            self.episode_cb, 10)

    def episode_cb(self, msg):
        self.recent_results.append(msg.success)
        self.episodes_in_stage += 1

        self.maybe_advance()
        self.publish_next_task()
        self.publish_progress()

    def maybe_advance(self):
        if self.episodes_in_stage < self.get_parameter('min_episodes_per_stage').value:
            return
        if len(self.recent_results) < 50:
            return

        success_rate = sum(self.recent_results) / len(self.recent_results)
        threshold = self.get_threshold_for_stage(self.stage)
        if success_rate >= threshold:
            self.stage = min(self.stage + 1, 7)
            self.recent_results.clear()
            self.episodes_in_stage = 0
            self.get_logger().info(f'Advanced to stage {self.stage}')

    def publish_next_task(self):
        # 10% chance: replay easier
        replay_prob = self.get_parameter('replay_easier_prob').value
        if random.random() < replay_prob and self.stage > 1:
            stage = random.randint(1, self.stage - 1)
        else:
            stage = self.stage

        task = self.generate_task_params(stage)
        self.task_pub.publish(task)
```

11.4 Configuration

```
# curriculum.yaml
curriculum_manager:
  ros__parameters:
    mode: "manual" # manual/algmm/random
    min_episodes_per_stage: 500
    replay_easier_prob: 0.1
    advance_thresholds: [0.9, 0.85, 0.8, 0.75, 0.75, 0.7]
    max_stages: 7
    enable_comprehensive_eval: true
    eval_frequency_episodes: 1000
    use_ewc: false
    ewc_lambda: 100.0
```

§ 12

Open Questions & Audience Explanations

12.1 Open Research Questions

RQ1: POET-STYLE COEVOLUTION

Tasks και agents coevolve. Tasks adapt σε agent abilities, agent learns από novel tasks.

RQ2: CROSS-DOMAIN CURRICULUM

Train σε one domain (Gazebo offices), transfer σε different (hospitals).

RQ3: LIFELONG LEARNING

Continue learning after deployment, χωρίς catastrophic forgetting.

RQ4: ADVERSARIAL CURRICULUM

Adversary generates increasingly hard tasks. Σύνδεση με C25.

RQ5: META-CURRICULUM LEARNING

Learn how to design curricula. Higher-order learning.

12.2 Audience Explanations

Παιδί 12 ετών

«Όταν μαθαίνεις ποδήλατο, δεν αρχίζεις σε αυτοκινητόδρομο. Πρώτα σε γρασίδι (αν πέσεις, δεν πονάει). Μετά σε ήσυχο πεζοδρόμιο. Μετά σε δρόμο. Έδωσα στο ρομπότ μου το ίδιο: 7 στάδια από εύκολο σε δύσκολο. Όταν τελειώνει το ένα, προχωράει στο επόμενο. Το ρομπότ μαθαίνει 5-6 φορές πιο γρήγορα από το να ξεκινήσει κατευθείαν στο δύσκολο.»

Φίλος σε Καφέ

«Curriculum learning: αντί ένα PPO/TD3 agent να ξεκινάει σε full deployment complexity (δύσκολη convergence, sparse rewards), σταδιακή εκπαίδευση. 7 stages: open arena → corridors → static obstacles → humans → ethical zones → noise → full. Auto-advance όταν τοπικό success > threshold. Critical: CBF safety shield (C18) σε όλα τα stages — δεν χαλαρώνει safety. Result: 5-6× sample efficiency (10M σε 50M+). Bonus: agent learns transferable skills, καλύτερο sim2real (0.79 vs 0.22 flat).

Καθηγητής

«Curriculum learning (Bengio 2009, Elman 1993) με optimization landscape interpretation. Sequential warm-starting: θ_k^* good initialization για harder problem $k+1$. Continuation method framework (Eq 2.3).

Implementation: 7-stage manual curriculum (open arena → full DynNav) με auto-advance σε 75-90% thresholds. ALP-GMM (Portelas 2019) automatic alternative. 10% replay sampling για catastrophic forgetting prevention. CBF (C18) enforced σε όλα τα stages.

Empirical: 5-6× sample efficiency (10M vs 50M+). Wrong curriculum order is catastrophic (0.41 vs 0.82). Sim2real 0.79 vs 0.22 για flat. ALP-GMM slightly outperforms manual (0.83 vs 0.82) but με opacity cost.

Application paper. Engineering integration. Limitations: hand-design subjective, POET not implemented, real deployment limited.»

Reviewer Defensive

«Aware ότι curriculum learning (Bengio 2009), self-paced learning (Kumar 2010), ALP-GMM (Portelas 2019), POET (2019) είναι established. No curriculum algorithm novelty.

Συνεισφορά: application paper. (1) Hand-designed 7-stage curriculum για mobile robotics. (2) Empirical comparison manual vs ALP-GMM vs reverse vs random. (3) CBF safety integration σε όλα τα stages. (4) Quantification of catastrophic forgetting mitigation strategies.

Limitations: (a) hand-design subjective. (b) POET not implemented. (c) Real deployment partial. (d) No comparison με recent self-play approaches.

Targeted: CoRL, NeurIPS, ICLR, ICRA RL Track.»

Elevator

«7-stage curriculum που εκπαιδεύει RL agent (PPO/TD3) από easy σε hard scenarios. CBF safety σε όλα τα stages. Result: 5-6× sample efficiency vs flat training (10M vs 50M+ steps), καλύτερο sim2real (0.79 vs 0.22).»

12.3 Summary Table

Audience	Time	Key Message
Παιδί	60s	«Πρώτα γρασίδι, μετά πεζοδρόμιο, μετά δρόμος»
Φίλος	90s	«7-stage curriculum + CBF safety + 5× sample efficiency»
Καθηγητής	3min	«Continuation method + ALP-GMM + catastrophic forgetting analysis»
Reviewer	2min	«Application paper, curriculum + CBF, empirical comparison»
Elevator	30s	«5-6× sample efficiency για mobile robot RL»



Closing Remarks

Τι Έμαθα

Το βαθύ insight: **η order σε learning δεν είναι ουvertibal**. Reverse curriculum (hard → easy) is catastrophic (0.41 vs 0.82). Standard order (easy → hard) works. ALP-GMM (continuous adaptive) slightly better. Wrong order is dramatically worse than no curriculum. Αυτό αντικατοπτρίζει βιολογική intuition: gradual scaffolding είναι essential, όχι decorative.

Επίσης: **curriculum is bridging language between human teaching pedagogy και ML training**. Όταν εξηγείς σε ένα student, αρχίζεις από prerequisites. Curriculum learning φέρνει αυτή την principle σε RL. Είναι ωραίο ότι ML αρχικά θεωρήθηκε «pure mathematical optimization» αλλά τώρα ενσωματώνει pedagogical insights.

Σύνδεση με Επόμενο

Στο Chapter 23 θα δούμε 3D Gaussian Splatting. Σύνδεση: το curriculum που είδαμε εκπαιδεύει σε procedural environments. Σε C23 +C24 (NeRF), βλέπουμε advanced 3D scene representations που μπορούν να γίνουν part του training procedural generator — high-fidelity synthetic worlds για curriculum stages.

ΣΥΝΔΕΣΗ ΜΕ ΕΠΟΜΕΝΑ

- **C23 (Gaussian Splatting)**: realistic curriculum environments
- **C24 (NeRF)**: alternative dense representation
- **C21 (PPO)**: agent being curricularly trained
- **C25 (Adversarial)**: adversarial curriculum extension
- **C13 (World Model)**: world model για curriculum env generation

— Τέλος Κεφαλαίου 22 από 26 —

Συνέχισε με: **Chapter 23 — 3D Gaussian Splatting**